

AI LearnMate: An Intelligent Personalized Learning Assistant

Kirti Kushwaha (524110025), Amisha Rai (524110045), Tejaswinee Padhan (524110043)

Department of Computer Application

National Institute of Technology Kurukshetra, Kurukshetra, India

Submitted to: Dr. Sarika Jain

Abstract—AI LearnMate is a web-based personalized learning assistant designed to improve how students process, organize, and review academic study material. The system accepts PDF notes, extracts readable text, generates student-friendly summaries through Google Gemini, and represents important concepts as a knowledge graph. Its backend is implemented with Flask, while Neo4j stores structured concept nodes and relationships for graph-oriented retrieval and reasoning. The project demonstrates a practical neuro-symbolic approach in which neural language generation is grounded by symbolic knowledge representation. By combining PDF processing, summarization, graph traversal, topic disambiguation, quiz support, and study-path generation, AI LearnMate provides an extensible foundation for intelligent educational technology.

Index Terms—AI LearnMate, personalized learning, neuro-symbolic AI, knowledge graph, Neo4j, Flask, Gemini, educational technology.

I. INTRODUCTION

Digital learning resources have become abundant, but students often struggle to convert long lecture notes and PDF documents into structured, memorable knowledge. Conventional study workflows are usually passive: learners read documents, prepare notes manually, and switch between disconnected tools for revision, assessment, and planning. This creates information overload and reduces the learner's ability to identify conceptual dependencies.

AI LearnMate addresses this problem by combining large-language-model capabilities with graph-based knowledge representation. Instead of acting only as a document summarizer, the system is designed as an academic assistant that can extract relevant concepts, organize them into a relationship-aware graph, and support follow-up learning activities such as quizzes and study planning.

A. Problem Statement

Students require a system that can reduce lengthy notes into understandable summaries, reveal the conceptual structure hidden inside study materials, and guide revision in a personalized way. A simple database-backed note storage application is insufficient because academic concepts are highly connected: one concept may be part of a topic, related to another concept, or required before a later concept can be understood.

B. Objectives

The objectives of AI LearnMate are to provide AI-assisted PDF summarization, construct a graph representation of extracted academic concepts, enable topic-focused study paths, support quiz-based self-assessment, and offer a clean web interface for students. These objectives are implemented using a Flask application layer, the Google GenAI SDK for language generation, and Neo4j for graph persistence.

II. SYSTEM OVERVIEW

The application follows a layered architecture. The presentation layer uses HTML, Jinja2 templates, and CSS. The application layer uses Python with Flask routes and service modules. The AI layer communicates with Gemini through the Google GenAI client. The knowledge layer uses Neo4j to store concepts and semantic relationships, while the PDF processing layer extracts text from uploaded documents using PyMuPDF.

TABLE I: AI LEARNMATE TECHNOLOGY STACK

Component	Technology	Responsibility
Backend	Flask, Python	Routing, templates, request handling
AI Engine	Google GenAI / Gemini	Summary and learning-content generation
Knowledge Base	Neo4j	Concept nodes and graph relationships
PDF Processing	PyMuPDF	Text extraction from uploaded notes
Graph Logic	NetworkX and Neo4j service layer	Concept graph construction and traversal
Frontend	HTML5, Jinja2, CSS3	Student-facing web interface

A. Neuro-Symbolic Design

The system applies neuro-symbolic AI by pairing neural text understanding with symbolic graph storage. Gemini is used for flexible natural-language processing, while Neo4j records concepts as explicit nodes and relationships. This pairing improves interpretability because the system can expose how concepts are connected rather than returning only free-form generated text.

B. Knowledge Graph Representation

The knowledge graph represents extracted study concepts as nodes and uses relationships such as RELATED_TO, BEFORE, AFTER, and topic-focused links to support traversal. This model is more natural for educational knowledge than a purely tabular design because prerequisites, dependencies, and semantic associations can be queried directly.

III. KEY FEATURES

A. PDF Upload and Text Extraction

Students upload PDF notes through the web interface. The application validates the file type, stores the document in the upload directory, and uses PyMuPDF to extract page-level text. The extracted text becomes the input for summary generation and downstream concept processing.

B. AI-Powered Summarization

The summarization module sends extracted note content to Gemini with an educational prompt. The resulting summary is written in accessible language so that students can quickly revise long documents. Error handling is included for missing API configuration, empty inputs, and AI service timeouts.

C. Concept Graph and Study Path

After a summary is generated, the system extracts key multi-word study concepts, constructs a graph, and stores the graph in Neo4j. The study-path feature can fetch graph data and present an ordered view of topic relationships, helping students understand what to review before moving to more advanced material.

D. Quiz Support

AI LearnMate also includes quiz support for self-assessment. Quiz state is maintained by the application and can be connected to generated topics and summaries. This provides a natural extension from reading and summarization into active recall.

IV. SYSTEM DESIGN AND DATA FLOW

The core flow begins when a learner uploads a PDF. The Flask route validates and saves the file, the PDF service extracts text, the Gemini service generates an explanation or summary, the concept service extracts important concepts, and the Neo4j service persists the graph. The frontend then renders the result through Jinja2 templates.

Fig. 1. Summarization and graph-construction pipeline.

Step	Operation	Output
1	Upload PDF	Stored study note
2	Extract text	Raw academic content
3	Generate summary	Student-friendly explanation
4	Extract concepts	Concept list and graph edges
5	Save to Neo4j	Queryable knowledge graph
6	Render dashboard	Summary, graph, quiz and study controls

A. Application Routes

The main routes include the home page, upload route, graph view, topic selection, quiz route, study-path route, about page, and contact page. This route structure separates user-facing workflows while allowing shared service modules to handle AI generation, concept extraction, and database operations.

TABLE II: SELECTED APPLICATION ROUTES

Route	Purpose
/	Home page and project entry point
/upload	PDF upload and validation
/graph	Knowledge graph visualization
/topic-options	Ambiguous topic clarification
/quiz	Quiz interaction
/study-path	Learning path display

B. Data Model

The graph model centers on Concept nodes and directed relationships that capture ordering and semantic association. Neo4j is suitable for this structure because Cypher can express multi-hop traversals and neighborhood queries concisely, which is essential when generating learning paths from connected academic concepts.

V. IMPLEMENTATION HIGHLIGHTS

The implementation is organized into routes, services, models, templates, static assets, and utility functions. The Gemini service centralizes API configuration and guarded calls. The concept service cleans candidate terms, extracts multi-word concepts from generated summaries, and builds graph data. The Neo4j service manages driver creation, connection checks, graph writes, and graph reads.

The project also uses environment variables for sensitive configuration. This keeps API keys and database credentials

outside the main source code. The upload workflow uses Werkzeug's filename sanitization to reduce unsafe file-name handling during PDF storage.

TABLE III: FUNCTIONAL TESTING SUMMARY

Test Case	Expected Result	Status
Valid PDF upload	File stored and processed	Pass
Non-PDF upload	Rejected with user feedback	Pass
Gemini summary request	Summary generated or handled error	Pass
Neo4j connection	Status reflected in interface	Pass
Graph generation	Concept nodes and edges created	Pass
Quiz route	Quiz page rendered	Pass

VI. AI AND MACHINE LEARNING ASPECTS

A. Natural Language Processing

The system uses Gemini for natural-language tasks including summarization, simplification, and learning-oriented content generation. These capabilities allow the application to transform unstructured PDF text into concise explanations that are easier for students to revise.

B. Reasoning Through Graph Structure

Reasoning in AI LearnMate is supported by the symbolic graph layer. Once concepts are stored as nodes and relationships, the system can identify related topics, retrieve connected concepts, and form ordered study paths. This provides a degree of transparency that pure text generation alone cannot offer.

C. Topic Disambiguation

A learning assistant must handle ambiguous user input. A topic such as "tree" may refer to a data structure, a biological organism, or a general diagram depending on context. AI LearnMate includes an ambiguity-handling layer that compares the requested topic with available concepts and returns clarification options when necessary. This helps prevent the system from generating a study path for the wrong interpretation of a term.

D. Educational Value of the AI Layer

The AI component is not treated as an isolated chatbot. It is integrated with the student workflow so that generated summaries, concepts, quizzes, and study paths support each other. The summary reduces cognitive load, the graph exposes conceptual structure, and the quiz feature encourages active recall. Together, these components support comprehension, retention, and revision planning.

VII. EVALUATION AND DISCUSSION

The project was evaluated from a functional and architectural perspective. Functional evaluation focused on whether the main workflows operated as expected: uploading notes, extracting text,

producing AI summaries, constructing concept graphs, loading graph data from Neo4j, displaying study paths, and rendering quiz pages. Architectural evaluation focused on whether the selected technologies worked together coherently for an AI-driven learning system.

The Flask-based architecture proved suitable for rapid development because each major concern could be separated into a service module. PDF extraction, AI calls, concept processing, Neo4j operations, quiz behavior, and study-path construction are implemented as distinct responsibilities. This separation makes the system easier to debug and provides a clear path for future improvement.

TABLE IV: MODULE-LEVEL RESPONSIBILITIES

Module	Primary Role	Contribution
main_routes.py	Request routing	Connects user actions to services

Module	Primary Role	Contribution
pdf_service.py	PDF text extraction	Transforms uploaded notes into raw text
gemini_service.py	AI access	Creates summaries and learning content
concept_service.py	Concept processing	Builds graph-ready topic data
neo4j_service.py	Graph persistence	Stores and retrieves concept relationships
study_service.py	Study path logic	Prepares ordered learning guidance

A. Functional Observations

The most important observation is that the system benefits from combining generative and structured components. Gemini can summarize and explain content fluently, but the graph layer gives the project a memory structure that can be queried. Neo4j therefore serves as more than a storage engine; it becomes the symbolic representation through which the learning assistant can reason about conceptual connections.

The dashboard-centered workflow also improves usability. A student can upload notes, request summaries, inspect related concepts, and move toward quiz or study-path activities from the same application. This is valuable because educational tools often fail when they require students to manually connect outputs from separate systems.

B. Comparison with Conventional Systems

A conventional notes application stores files and perhaps allows searching by keyword. AI LearnMate adds interpretation and organization. Compared with a relational database design, the graph design better represents relationships such as sequence, dependency, and similarity. Compared with a pure LLM interface, the graph design gives the system an explicit structure that can be reused across multiple features.

TABLE V: COMPARISON WITH COMMON LEARNING TOOLS

Capability	Conventional Notes Tool	AI LearnMate
PDF handling	Stores files	Extracts and processes text
Summary	Manual preparation	AI-generated explanation
Concept structure	Mostly absent	Graph-based relationships
Revision support	User-managed	Quiz and study-path support
Reasoning model	Keyword or folder based	Neuro-symbolic representation

VIII. SECURITY AND RELIABILITY CONSIDERATIONS

Because AI LearnMate handles student-uploaded academic material and external AI calls, reliability and security must be considered even at prototype stage. The implementation avoids hard-coding sensitive configuration by loading API and database values from environment variables. This reduces accidental exposure of secrets in source files and supports safer deployment practices.

The upload workflow applies basic validation by accepting PDF files and using secure filename handling. These protections are important because file upload features are common sources of risk in web applications. A production deployment should extend this foundation with file-size limits, malware scanning, stricter MIME validation, and user-specific storage boundaries.

A. AI Service Reliability

External AI APIs can fail because of missing keys, network errors, rate limits, long inputs, or service timeouts. The Gemini service therefore performs guarded calls and returns controlled errors rather than allowing failures to crash the application. This reliability layer is essential for a learning tool because students need clear feedback when a summary cannot be generated.

B. Data Consistency

Graph consistency is important when concepts are regenerated or updated. The Neo4j service manages concept writes so that stale or duplicate relationships can be controlled. Future versions can strengthen this further through explicit document identifiers, versioned graph snapshots, and validation rules for relationship types.

IX. CHALLENGES AND LIMITATIONS

The main technical challenges include handling variable PDF text quality, extracting clean concepts from generated summaries, mapping flexible natural language into structured graph data, and maintaining useful behavior when external AI or database services are unavailable. The system currently works best with text-selectable PDFs; scanned documents require OCR support.

Large documents may also require chunking to stay within model limits and to improve summary quality. Future versions can add richer authentication, persistent quiz history, learning analytics, OCR, and more formal validation of generated graph relationships.

A. PDF Quality Limitations

PDF documents vary widely in structure. A clean lecture note with selectable text can be processed accurately, while scanned pages, mathematical notation, tables, and diagrams may not extract into meaningful plain text. OCR integration and layout-aware extraction would improve the system's ability to handle a wider range of academic documents.

B. LLM Output Variability

LLM responses are powerful but not perfectly deterministic. Summary style, concept wording, and relationship hints may vary

across calls. Prompt engineering reduces this variability, but production systems should include schema validation and post-processing rules before committing generated content into the graph.

X. FUTURE SCOPE

AI LearnMate can be extended in several directions. The most immediate extension is persistent user-level learning history so that the system can remember completed quizzes, weak topics, preferred subjects, and repeated mistakes. This would allow recommendations to become increasingly personalized over time.

Another important extension is adaptive study planning. Once the system knows prerequisite relationships and quiz performance, it can recommend a sequence of topics that balances difficulty, urgency, and learner progress. This would transform the platform from a

summarization tool into a guided learning environment.

The knowledge graph can also be enriched with external academic ontologies, textbook indexes, and course syllabi. Such enrichment would help the system compare a student's uploaded notes with a broader subject map and identify missing concepts. In addition, graph visualization can be improved with interactive filtering, clustering, and path-highlighting features.

TABLE VI: FUTURE ENHANCEMENT ROADMAP

Enhancement	Expected Benefit
OCR for scanned PDFs	Supports image-based notes and handwritten scans
Chunked summarization	Improves handling of long study material
Persistent quiz analytics	Tracks weak topics and learning progress
Adaptive study planner	Recommends personalized revision sequences
Graph visualization filters	Improves exploration of large concept graphs
Exportable summaries	Allows students to save revision material

XI. USER INTERFACE AND LEARNING EXPERIENCE

The user interface of AI LearnMate is intentionally simple because the target users are students who need fast access to learning functions rather than a complex administrative tool. The application presents upload, graph, quiz, and study-path workflows through server-rendered pages. This keeps the interface responsive enough for a prototype and makes the learning flow understandable without requiring the student to configure technical parameters.

A typical learner begins from the home page, uploads a PDF, waits for the system to extract text, and then receives an AI-generated summary. From that point, the learner can inspect the graph representation, choose a topic, take a quiz, or open a study path. The design therefore follows the natural academic sequence of reading, organizing, recalling, and planning.

A. Dashboard-Oriented Interaction

The dashboard acts as the main control center. This is important because students should not need to remember where each output is stored. A dashboard-oriented design allows uploaded material, generated learning artifacts, and follow-up actions to remain connected. In future versions, the dashboard can also display progress indicators, recent quiz results, weak-topic alerts, and recommended next actions.

B. Graph Visualization as a Learning Aid

Graph visualization helps students see that subjects are not isolated lists of definitions. Concepts can be arranged as connected ideas, and those connections can reveal prerequisites or related areas. For example, in a data-structures topic, arrays, linked lists, stacks, queues, trees, and graphs can be displayed as related structures rather than independent chapters. This supports conceptual learning instead of rote memorization.

TABLE VII: LEARNING WORKFLOW SUPPORTED BY THE SYSTEM

Learning Stage	System Support	Student Benefit
Input	PDF upload and text extraction	Uses existing study material
Understanding	AI-generated summary	Reduces reading burden
Organization	Knowledge graph	Shows relationships among concepts
Practice	Quiz workflow	Encourages active recall
Planning	Study path	Guides ordered revision

XII. DEPLOYMENT AND MAINTAINABILITY

AI LearnMate is structured so that it can be maintained and extended without rewriting the entire application. The main Flask application initializes routes, template filters, Gemini startup checks, and error handlers. Functional behavior is then distributed into focused service files. This organization is suitable for a student project because each module can be tested and improved independently.

For local deployment, the application requires Python dependencies, a Gemini API key, and Neo4j connection details. These values are loaded from environment configuration, allowing the same codebase to run across machines with different credentials. A production deployment would additionally need HTTPS, user authentication hardening, upload quotas, logging, backup policies, and monitoring for AI-service failures.

A. Maintainability Benefits

The modular structure creates a clear separation between web routing and core logic. For example, the PDF extraction strategy can be changed without modifying the quiz route, and the LLM provider can be replaced without changing the graph visualization template. This separation also supports future migration from a prototype into a fuller application.

B. Ethical and Educational Considerations

Educational AI systems should assist learning rather than replace effort. AI LearnMate is therefore best viewed as a support tool that helps students understand and revise material. The system should encourage students to verify summaries against source notes, practice through quizzes, and use study paths as guidance rather than as a substitute for independent reasoning.

The project also raises important questions about privacy and responsible AI use. Uploaded notes may contain personal annotations or institutional material, so future deployments should define clear data-retention policies. AI-generated summaries should be presented with the expectation that students and teachers may review them for correctness.

XIII. CONCLUSION

AI LearnMate demonstrates how modern AI services can be combined with graph databases to build an intelligent educational

assistant. The project moves beyond simple note storage by offering AI-generated summaries, concept extraction, knowledge graph construction, quiz support, and study-path generation. Its neuro-symbolic architecture makes it extensible for future adaptive learning features while preserving an interpretable representation of academic concepts.

The completed prototype shows that even a compact student-built system can apply advanced AI ideas to a practical educational problem. Its strongest contribution is the integration of LLM-based language understanding with Neo4j-backed symbolic representation. This makes the platform a meaningful foundation for future work in personalized learning, explainable educational AI, and graph-driven academic assistance.

ACKNOWLEDGMENT

The authors express sincere gratitude to their guide and supervisor, Satendar Sir, for valuable guidance, encouragement, and support throughout the development of AI LearnMate. The authors also thank the Department of Computer Application, National Institute of Technology Kurukshetra, faculty members, classmates, friends, and families for their support and feedback.

REFERENCES

- [1] Google, "Google GenAI Python SDK," 2024. [Online]. Available: <https://github.com/googleapis/python-genai>
- [2] Neo4j, Inc., "Neo4j Graph Database Documentation," 2024. [Online]. Available: <https://neo4j.com/docs/>
- [3] Pallets Projects, "Flask Documentation," 2024. [Online]. Available: <https://flask.palletsprojects.com/>
- [4] Artifex Software, "PyMuPDF Documentation," 2024. [Online]. Available: <https://pymupdf.readthedocs.io/>
- [5] NetworkX Developers, "NetworkX Documentation," 2024. [Online]. Available: <https://networkx.org/documentation/>
- [6] Pallets Projects, "Werkzeug Documentation," 2024. [Online]. Available: <https://werkzeug.palletsprojects.com/>
- [7] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.
- [8] G. Marcus and E. Davis, Rebooting AI: Building Artificial Intelligence We Can Trust. Pantheon Books, 2019.
- [9] J. F. Sowa, Knowledge Representation: Logical, Philosophical, and Computational Foundations. Brooks/Cole, 2000.
- [10] A. d'Avila Garcez, M. Gori, L. C. Lamb, L. Serafini, M. Spranger, and S. N. Tran, "Neural-symbolic learning and reasoning: A survey and interpretation," Neuro-Symbolic Artificial Intelligence, IOS Press, 2022.
- [11] L. C. Lamb, A. d'Avila Garcez, M. Gori, M. O. R. Prates, P. H. C. Avelar, and M. Y. Vardi, "Graph neural networks meet neural-symbolic computing: A survey and perspective," in Proc. Int. Joint Conf. Artificial Intelligence, 2020.
- [12] T. R. Gruber, "A translation approach to portable ontology specifications," Knowledge Acquisition, vol. 5, no. 2, pp. 199-220, 1993.
- [13] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. draft, 2024.
- [14] A. Vaswani et al., "Attention is all you need," in Proc. Advances in Neural Information Processing Systems, 2017.